

Apache Beam / Mlflow

Practical Session 3

Juan Ramírez
jramirezg@faculty.ie.edu

Apache Beam

Apache Beam (<https://beam.apache.org/>)



About

Get Started

Documentation ▾

Roadmap

Community

Contribute

Blog

Case Studies



Apache >

INTRODUCING APACHE BEAM

The Unified Apache Beam Model

The easiest way to do batch and streaming data processing.
Write once, run anywhere data processing for mission-critical production workloads.

📄 LINK TO GITHUB REPO

📘 LEARN MORE

JAVA QUICKSTART

PYTHON QUICKSTART

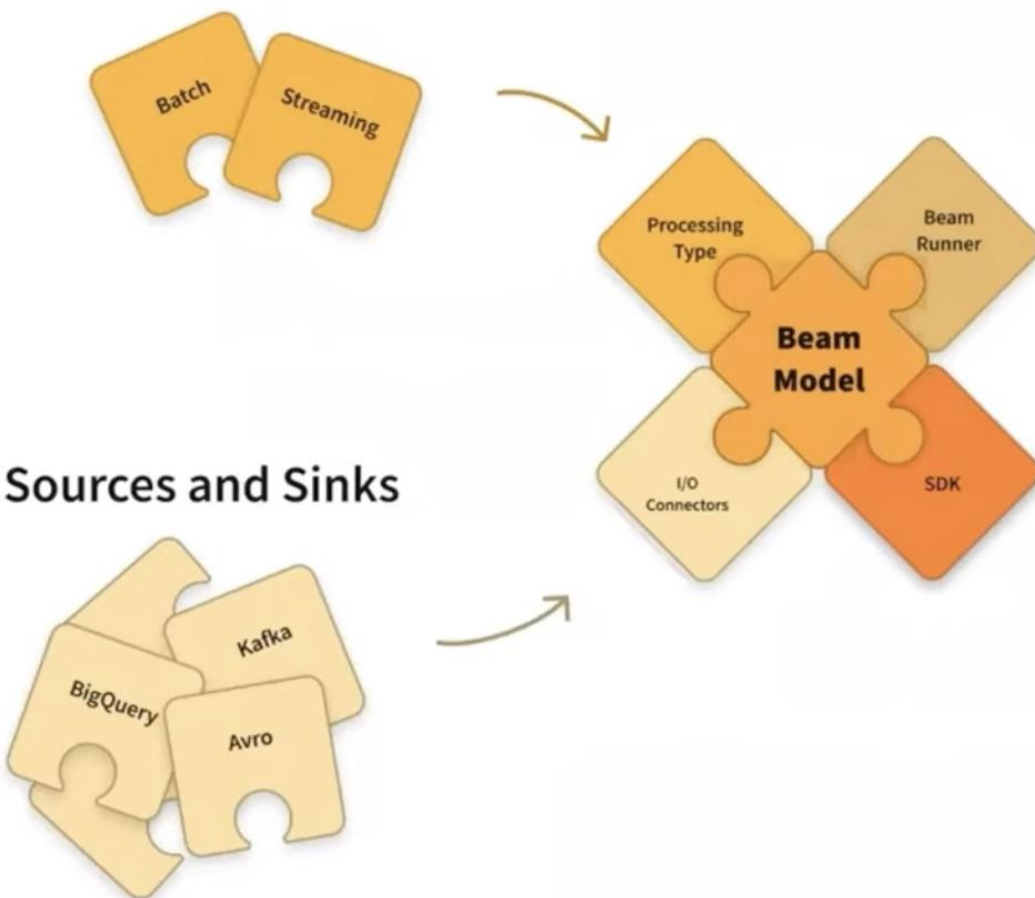
GO QUICKSTART

TRY PLAYGROUND

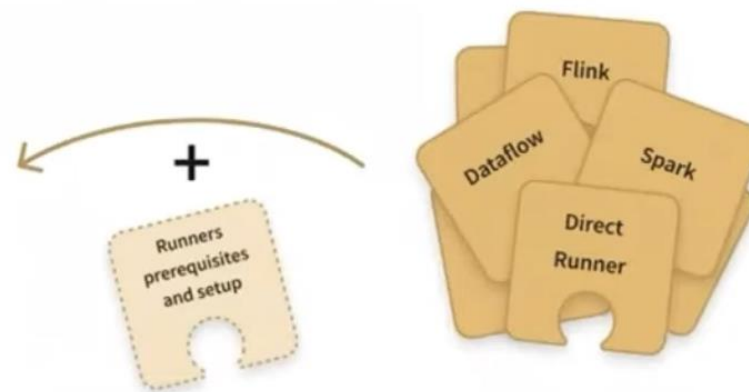
How Does It Work?

Apache Beam

Unified Programming Model



Execution Runners



Languages



Apache Beam

SDK Support Different Languages

Java

```
input.apply(  
  Sum.integersPerKey()  
)
```

Python

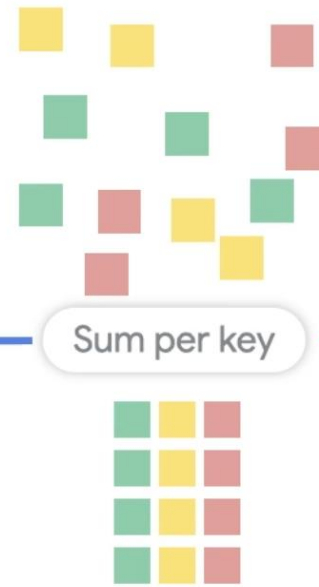
```
input | Sum.PerKey()
```

Go

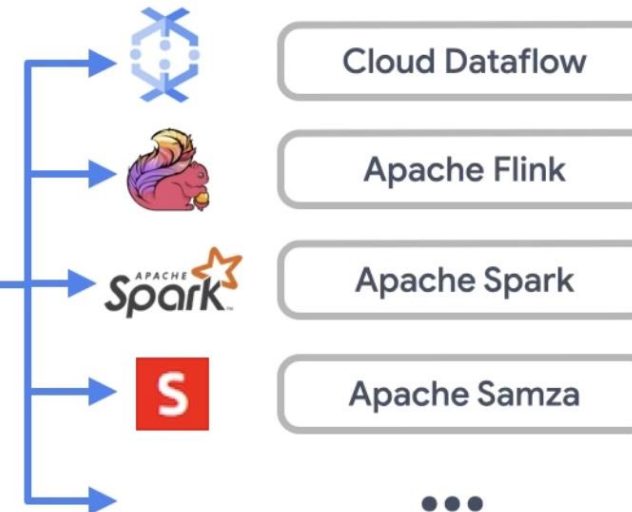
```
stats.Sum(s, input)
```

SQL

```
SELECT key, SUM(value)  
FROM input GROUP BY key
```



Support Different Execution Engines



Apache Beam Playground (<https://beam.apache.org/>)

The screenshot shows the Apache Beam website's main landing page. The navigation bar at the top includes links for 'beam', 'About', 'Get Started', 'Documentation', 'Roadmap', 'Community', 'Contribute', 'Blog', 'Case Studies', a search icon, an edit icon, and the 'Apache' logo. The main content area features a large orange banner with the text 'INTRODUCING APACHE BEAM' and 'The Unified Apache Beam Model'. Below this, it states 'The easiest way to do batch and streaming data processing. Write once, run anywhere data processing for mission-critical production workloads.' and provides a 'LINK TO GITHUB REPO' button. At the bottom, there are five buttons: 'LEARN MORE', 'JAVA QUICKSTART', 'PYTHON QUICKSTART', 'GO QUICKSTART', and 'TRY PLAYGROUND'. The 'TRY PLAYGROUND' button is highlighted with a blue border. A callout box with a blue border and a pointer to the button contains the text: 'You can try Apache Beam without any installation or setup: click on 'Try Playground''. Below the buttons, the text 'How Does It Work?' is visible.

beam About Get Started Documentation Roadmap Community Contribute Blog Case Studies Apache

INTRODUCING APACHE BEAM

The Unified Apache Beam Model

The easiest way to do batch and streaming data processing. Write once, run anywhere data processing for mission-critical production workloads.

LINK TO GITHUB REPO

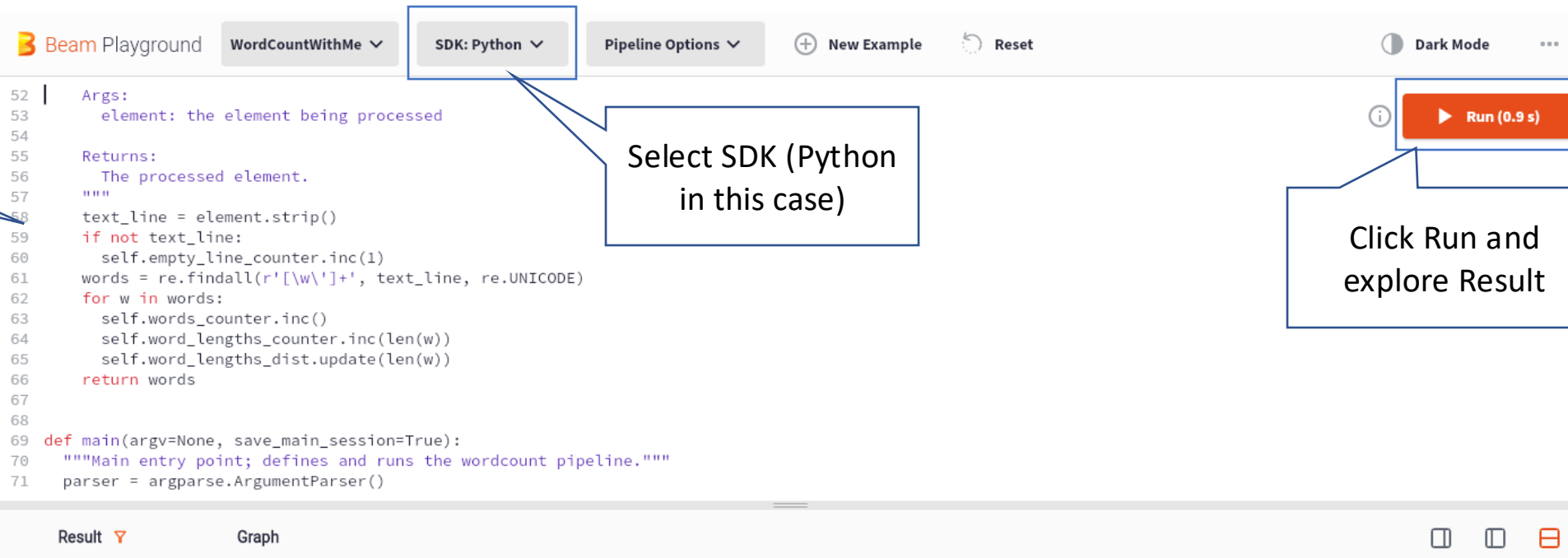
LEARN MORE JAVA QUICKSTART PYTHON QUICKSTART GO QUICKSTART TRY PLAYGROUND

How Does It Work?

You can try Apache Beam without any installation or setup: click on 'Try Playground'

Apache Beam Playground (<https://beam.apache.org/>)

Pipeline example definition using selected SDK



The screenshot shows the Apache Beam Playground interface. At the top, there's a header with the Beam logo, a dropdown menu for 'WordCountWithMe', a dropdown menu for 'SDK: Python' (highlighted with a box and an annotation), a 'Pipeline Options' dropdown, and buttons for 'New Example' and 'Reset'. On the right, there's a 'Dark Mode' toggle and a 'Run (0.9 s)' button (highlighted with a box and an annotation). The main area displays a Python code snippet for a word count pipeline. Below the code, there are tabs for 'Result' and 'Graph'. The 'Result' tab is active, showing a table with columns 'word' and 'count'. The table contains three rows: 'the' with count 1, 'word' with count 1, and 'count' with count 1. The 'Graph' tab is also visible, showing a visual representation of the pipeline.

```
52 | Args:
53 |     element: the element being processed
54 |
55 | Returns:
56 |     The processed element.
57 | """
58 | text_line = element.strip()
59 | if not text_line:
60 |     self.empty_line_counter.inc(1)
61 | words = re.findall(r'[\w\']+ ', text_line, re.UNICODE)
62 | for w in words:
63 |     self.words_counter.inc()
64 |     self.word_lengths_counter.inc(len(w))
65 |     self.word_lengths_dist.update(len(w))
66 | return words
67 |
68 |
69 | def main(argv=None, save_main_session=True):
70 |     """Main entry point; defines and runs the wordcount pipeline."""
71 |     parser = argparse.ArgumentParser()
```

Select SDK (Python in this case)

Click Run and explore Result

Apache Beam Playground [\(https://beam.apache.org/\)](https://beam.apache.org/)


 Beam Playground

WordCountWithMe ▾

SDK: Python ▾

Pipeline Options ▾

+ New Example

↺ Reset

Dark Mode

⋮

```

52     Args:
53         element: the element being processed
54
55     Returns:
56         The processed element.
57     """
58     text_line = element.strip()
59     if not text_line:
60         self.empty_line_counter.inc(1)
61     words = re.findall(r'[\w\']+ ', text_line, re.UNICODE)
62     # ... do something ...
  
```

Result ▾

Graph ●

2022-07-12 11:50:32,300 [INFO]

Default Python SDK image for environment is apache/beam_python3.7_sdk:2.40.0

2022-07-12 11:50:32,418 [INFO]

===== <function annotate_downstream_side_inputs at 0x7f3b0be3a8c0> =====

2022-07-12 11:50:32,418 [INFO]

===== <function fix_side_input_pcoll_coders at 0x7f3b0be3a9e0> =====

2022-07-12 11:50:32,419 [INFO]

===== <function pack_combiners at 0x7f3b0be3aef0> =====

2022-07-12 11:50:32,420 [INFO]

===== <function lift_combiners at 0x7f3b0be3af80> =====

2022-07-12 11:50:32,420 [INFO]

===== <function expand_sdf at 0x7f3b0be3b170> =====

2022-07-12 11:50:32,421 [INFO]

===== <function expand_gbk at 0x7f3b0be3b200> =====

2022-07-12 11:50:32,422 [INFO]

===== <function sink_flattens at 0x7f3b0be3b320> =====

2022-07-12 11:50:32,422 [INFO]

===== <function greedily_fuse at 0x7f3b0be3b3b0> =====

2022-07-12 11:50:32,425 [INFO]

===== <function read_to_impulse at 0x7f3b0be3b440> =====

2022-07-12 11:50:32,425 [INFO]

===== <function impulse_to_input at 0x7f3b0be3b4d0> =====

2022-07-12 11:50:32,425 [INFO]

===== <function sort_stages at 0x7f3b0be3b710> =====

2022-07-12 11:50:32,426 [INFO]

===== <function add_impulse_to_dangling_transforms at 0x7f3b0be3b830> =====

2022-07-12 11:50:32,426 [INFO]

===== <function setup_timer_mapping at 0x7f3b0be3b680> =====

2022-07-12 11:50:32,427 [INFO]

===== <function populate_data_channel_coders at 0x7f3b0be3b7a0> =====

2022-07-12 11:50:32,432 [INFO]

Creating state cache with size 100

2022-07-12 11:50:32,433 [INFO]

Created Worker handler <apache_beam.runners.portability.fn_api_runner.worker_handlers.EmbeddedWorkerHandler object at 0x7f3b100df2d0>

for environment ref_Environment_default_environment_2 (beam:env:embedded_python:v1, b'')

2022-07-12 11:50:33,667 [INFO]

Starting finalize_write threads with num_shards: 1 (skipped: 0), batches: 1, num_threads: 1

2022-07-12 11:50:33,769 [INFO]

Renamed 1 shards in 0.10 seconds.

2022-07-12 11:50:33,787 [INFO]

number of empty lines: 1663

2022-07-12 11:50:33,788 [INFO]

average word length: 4

Apache Beam Playground [\(https://beam.apache.org/\)](https://beam.apache.org/)

Beam Playground WordCountWithMe SDK: Python Pipeline Options New Example Reset Dark Mode

```
52 Args:
53     element: the element being processed
54
55 Returns:
56     The processed element.
57 """
58 text_line = element.strip()
59 if not text_line:
60     self.empty_line_counter.inc(1)
61 words = re.findall(r'[\w\']+ ', text_line, re.UNICODE)
62 return words
```

Run (0.9 s)

Result Graph



```
graph LR
    read --> PCollection3067
    PCollection3067 --> split
    split --> PCollection5433
    PCollection5433 --> pair_with_one
    pair_with_one --> PCollection3619
    PCollection3619 --> group
    group --> PCollection5284
    PCollection5284 --> count
    count --> PCollection2
```

MLFlow



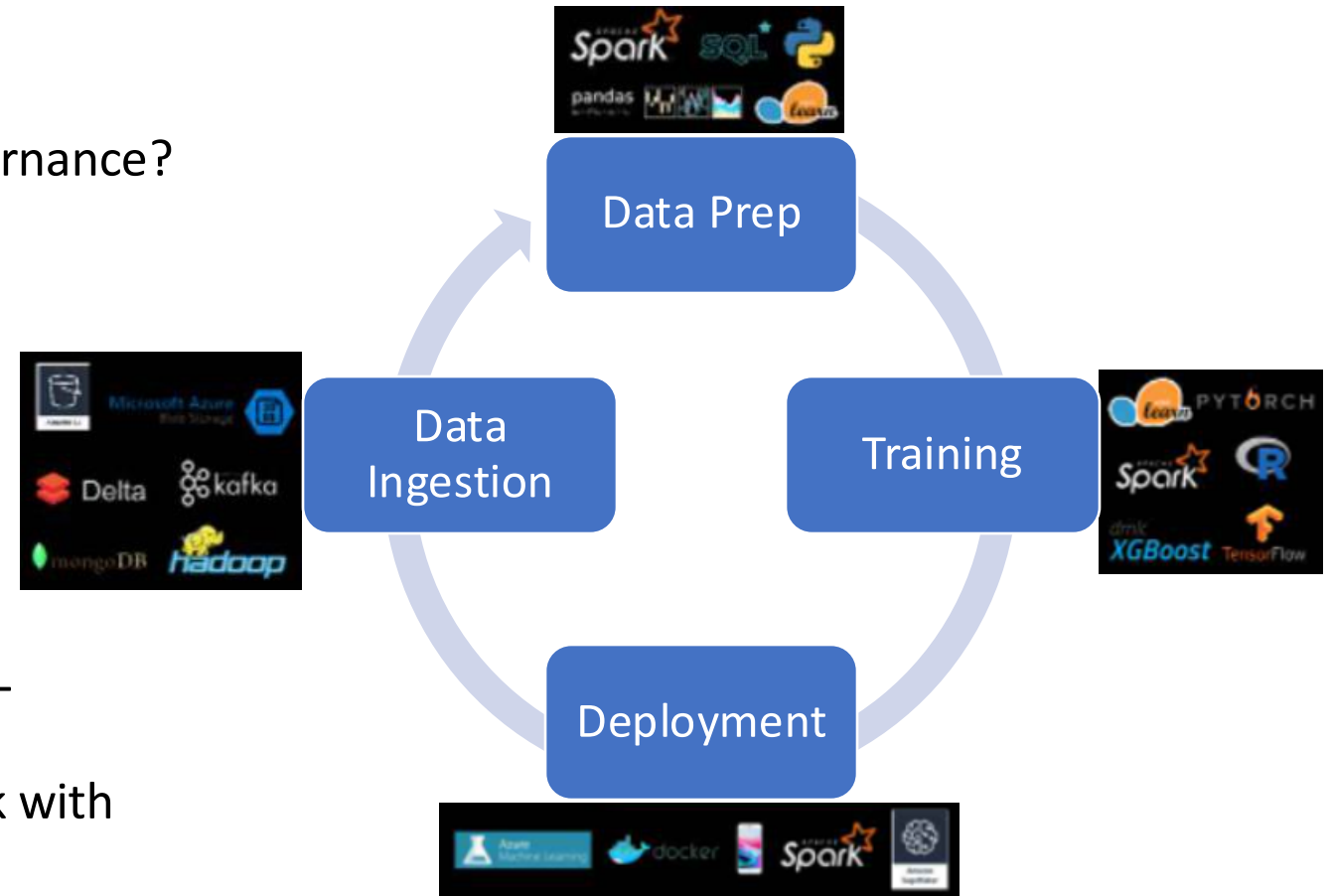
https://github.com/juanmaramirezgarcia/mlops_course_materials

What is Mlflow?

<https://www.mlflow.org/>



- A tool for all stages in the ML lifecycle?
- A tool that can scale with the project?
- A tool that supports model exchange and governance?
 - Complete lineage
 - Source code performance metrics
 - Who trained the model



Mlflow is an open source platform to manage ML development

- Lightweight APIs & abstractions that work with any ML library
- Designed with scalability in mind to be useful for 1 user or 1000+ person orgs
- Runs the same way anywhere (e.g. any cloud)

MLflow Components



MLflow Tracking

MLflow Tracking is an API and user interface component that records data about machine learning experiments and lets you query it.

MLflow Tracking supports Python, as well as various APIs like REST, Java API, and R API.

You can use this component to log several aspects of your runs

MLflow Projects

MLflow Projects let you package data science code in a reproducible and reusable way.

The projects component comes with an API as well as command-line tools designed for running projects. These features ensure you can chain projects together into machine learning workflows.API.

MLflow Models

MLflow Models let you package machine learning models in a format supported by many downstream tools.

You can add metadata to your MLflow models

Key Concepts in Tracking



- **Source**—can be the name of the file that launches the run. Alternatively, if you are using an MLflow project, it can be the name of the project and entry point of the run.
- **Code version**—when using an MLflow Project, this would be the **Git commit hash**.
- **Parameters**—can be any key-value input parameters you choose, as long as the values and the keys are both strings.
- **Artifacts**—are output files (in all formats). Artifacts let you record images, PNGs for example, **models** (such as pickled scikit-learn models), and data files such as Parquet files.
- **Start and end time**—lets you record the start and end time of your run.
- **Metrics**—let you record key-value metrics containing numeric values. It is possible to update each metric throughout the duration of a run. This lets you, for example, track how the loss function of the model is converging. Additionally, MLflow lets you visualize the full history of each metric.
- **Notes**

Mlflow Project Properties



- **Name**—you can choose a human-readable name for your projects.
- **Environment**—lets you define the software environment used to execute the entry points of the project, including all library dependencies required by project code.
- **Entry points**—lets you specify the commands you want to run inside the project, as well as information about the parameters. A project typically contains at least one entry point that users can call.

Notes:

You can use a Git URI to run your project (connect directly with the git repository that contains an Mlflow project).

Alternatively, you can use the MLflow run command-line tool to run the project from a local directory.

MLflow Model Properties



- **Model signature**—defines the schema of the inputs and outputs of your model. You can use either tensor-based or column-based model inputs and outputs. You can describe column-based inputs and outputs as a sequence of named columns and specify the MLflow data type. You can do the same for tensor-based inputs and outputs, and specify a numpy data type. Note that signatures are stored as JSON in the MLmodel file, along with additional model metadata.
- **Model input example**—this is an artifact that is an instance of a model input, stored together with your model. You can use either column-based or tensor-based inputs.

MLflow Model Registry



MLflow Model Registry provides an API and UI for centrally managing your models and their lifecycle. The registry provides model lineage, model versioning, annotations, and stage transitions.

Key features:

- **Registered model:** In MLflow, a registered model is one that has a unique name and metadata, contains model versions and transitional stages, and has a model lineage.
- **Model version:** A registered model can contain one or more model versions. When you register a new model in the registry, it is considered version 1. Any new model using the same name is added as a subsequent version.
- **Model stage:** For each model version, you can assign one stage, at any time. However, stages must be assigned according to the officially determined MLflow stages, such as staging, production, and archived. It is possible to transition a model version from one stage to another.
- **Annotations and descriptions:** MLflow lets you annotate the top-level model as well as each individual version, using markdown. You can add descriptions and any relevant information, such as algorithm descriptions, methodology, and dataset employed.